LAPORAN PRAKTIKUM
JOBSHEET 8 - INTEGRASI BASIS DATA POSTGRESQL DAN PDO



Disusun Oleh:

Nama	:	Mohammad Hafidz Rafi' Rabbani
NIM	:	254107020084
Kelas	:	2H
No. Absen	:	14
Repository	:	github.com/hafidzrafi/DnPWeb2026

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Daftar Isi

1. Tujuan 3

2. Praktikum 3

2.1. Percobaan 1: Perancangan Skema Database dan Migrasi DDL 3

2.1.1. 1.1 Struktur DDL Tabel Buku dan Anggota 3

2.1.2. Langkah-langkah 4

2.1.3. Pertanyaan 6

2.2. Percobaan 2: Pembuatan Modul Koneksi PDO (PHP Data Objects) 7

2.2.1. 2.1 Konfigurasi DSN dan Error Handling Berbasis Exception 7

2.2.2. Langkah-langkah 7

2.2.3. Pertanyaan 9

2.3. Percobaan 3: Penyisipan dan Pembacaan Data Menggunakan Prepared Statements 10

2.3.1. 3.1 Eliminasi Kerentanan SQL Injection dengan Prepared Statements 10

2.3.2. 3.2 Pembacaan Data dan Perenderan Tabel 10

2.3.3. Langkah-langkah 12

2.3.4. Pertanyaan 13

3. Tugas 14

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 8 14

1. Tujuan

- Memahami konsep dasar persistensi data menggunakan sistem manajemen basis data relasional (RDBMS) PostgreSQL.
- Menguasai teknik Data Definition Language (DDL) untuk perancangan skema tabel buku dan anggota pada berkas 01_buku_anggota.sql.
- Mengkonfigurasi koneksi database modern berbasis pustaka PHP Data Objects (PDO) pada berkas includes/koneksi.php.
- Menerapkan konfigurasi penanganan error berbasis PDOException serta fetch mode array asosiatif (PDO::FETCH_ASSOC).
- Menerapkan teknik Prepared Statements berparameter untuk memitigasi kerentanan keamanan SQL Injection secara mutlak.
- Memigrasikan pembacaan dan penyimpanan data aplikasi SIMPUS-Mini dari $_SESSION ke tabel relasional PostgreSQL.
- Menganalisis penegakan integritas data pada tingkat database melalui batasan PRIMARY KEY dan UNIQUE constraints.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: https://github.com/hafidzrafi/DnPWeb2026.

2. Praktikum

2.1. Percobaan 1: Perancangan Skema Database dan Migrasi DDL

Pada percobaan pertama, struktur persistensi data aplikasi SIMPUS-Mini dimigrasikan dari memori sementara sesi ke basis data PostgreSQL bernama `simpus_mini`.

2.1.1. 1.1 Struktur DDL Tabel Buku dan Anggota

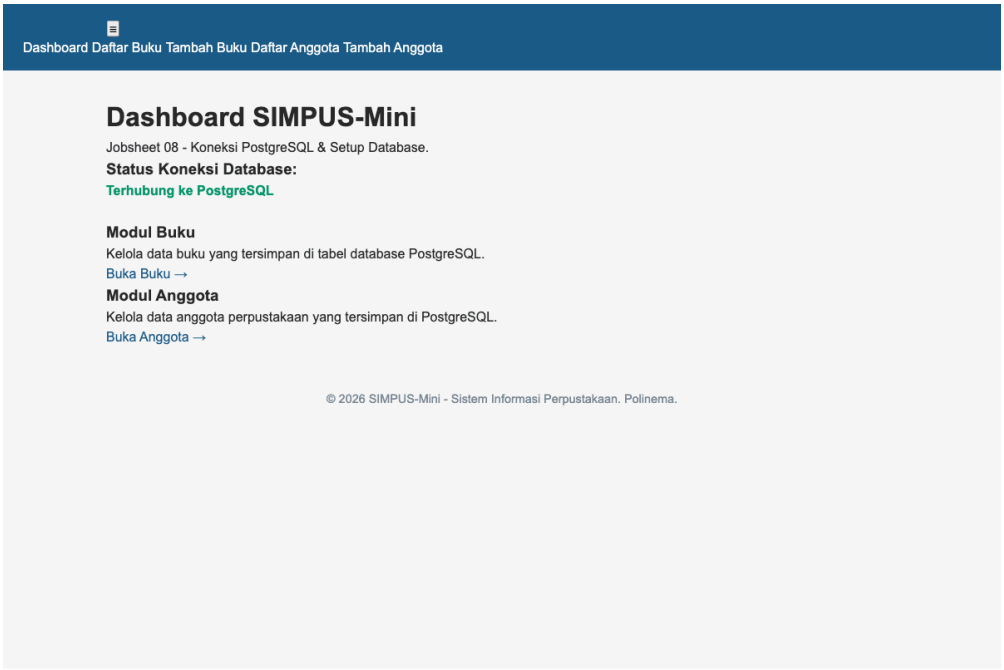
Skrip migrasi SQL `sql/01_buku_anggota.sql` dieksekusi untuk mengonstruksi dua tabel relasional utama:

```
CREATE TABLE IF NOT EXISTS buku (
    id SERIAL PRIMARY KEY,
    judul VARCHAR(255) NOT NULL,
    pengarang VARCHAR(255) NOT NULL,
    tahun INTEGER NOT NULL,
    isbn VARCHAR(50),
    stok INTEGER NOT NULL DEFAULT 0,
    kategori VARCHAR(50)
```

```
);

CREATE TABLE IF NOT EXISTS anggota (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(255) NOT NULL,
    no_anggota VARCHAR(50) NOT NULL UNIQUE,
    alamat VARCHAR(255),
    no_hp VARCHAR(30)
);
```

Penggunaan kolom `id SERIAL PRIMARY KEY` menjamin setiap baris memiliki kunci pengenal unik (**auto-increment**), sedangkan `no_anggota VARCHAR(50) NOT NULL UNIQUE` mencegah terjadinya nomor anggota ganda.



Gambar 1.1: Indikator Status Koneksi Database Berhasil Terhubung ke Layanan PostgreSQL Lokal.

2.1.2. Langkah-langkah

1. Membuat basis data `simpus_mini` menggunakan perintah CLI PostgreSQL.
2. Mengeksekusi skrip `sql/01_buku_anggota.sql` melalui command `psql -d simpus_mini -f sql/01_buku_anggota.sql`.
3. Memverifikasi struktur skema tabel melalui perintah periksa relasi `\dt` pada terminal psql.

4. Memeriksa status keterhubungan database pada antarmuka Dashboard `index.php`.

2.1.3. Pertanyaan

1. Mengapa migrasi dari `$_SESSION` ke PostgreSQL merupakan langkah arsitektural yang krusial?
    - Variabel `$_SESSION` bersifat sementara (**in-memory**) dan terisolasi per pengguna. Data akan musnah saat peramban ditutup atau server di-restart, serta tidak dapat diakses bersama antar-pengguna. PostgreSQL menyediakan persistensi data permanen, konkurensi multi-pengguna yang konsisten, serta integritas transaksi berbasis prinsip ACID.

2.2. Percobaan 2: Pembuatan Modul Koneksi PDO (PHP Data Objects)

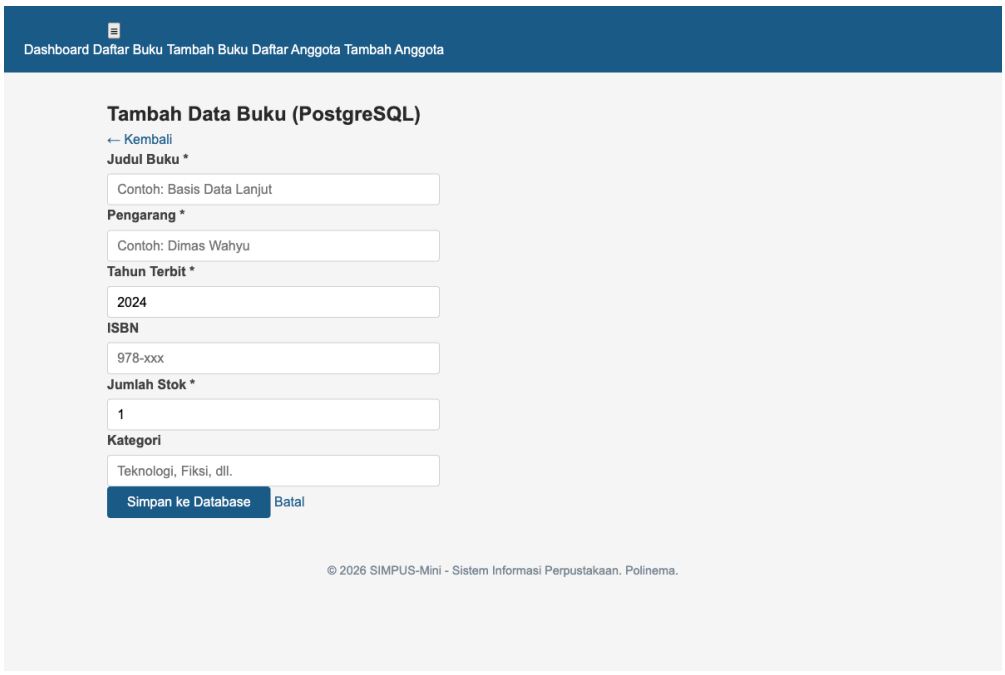
Percobaan kedua membangun modul konektivitas database terpusat pada berkas `includes/koneksi.php` menggunakan pustaka PDO.

2.2.1. 2.1 Konfigurasi DSN dan Error Handling Berbasis Exception

PDO diinisialisasi menggunakan string koneksi DSN (**Data Source Name**) PostgreSQL dan dikonfigurasi untuk melempar pengecualian `PDOException`:

```
$host = "localhost";
$port = "5432";
$db   = "simpus_mini";
$user = "postgres";
$pass = "postgres";

try {
    $pdo = new PDO("pgsql:host=$host;port=$port;dbname=$db", $user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    $db_error = $e->getMessage();
}
```



Gambar 1.2: Formulir Tambah Data Buku yang Terintegrasi dengan Lapisan Penyimpanan PostgreSQL.

2.2.2. Langkah-langkah

1. Menyusun berkas modul `includes/koneksi.php` dengan parameter koneksi host, port, dbname, user, dan password.

2. Mengatur atribut penanganan galat `PDO::ATTR_ERRMODE` ke `PDO::ERRMODE_EXCEPTION`.
3. Mengatur default pengambilan data ke mode asosiatif `PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC`.
4. Menguji instansiasi objek PDO melalui blok try...catch.

2.2.3. Pertanyaan

1. Apa keuntungan menggunakan PDO dibandingkan ekstensi spesifik database seperti `pg_connect()`?

    - PDO menyediakan antarmuka akses data seragam yang portabel antar-berbagai DBMS (PostgreSQL, MySQL, SQLite) tanpa perlu mengubah logika pemanggilan query aplikasi jika terjadi pergantian engine database. Selain itu, PDO mendukung prepared statements berparameter native dan error handling berbasis exception yang tangguh.

2.3. Percobaan 3: Penyisipan dan Pembacaan Data Menggunakan Prepared Statements

Percobaan ketiga mengimplementasikan pembacaan data (`SELECT`) dan penyimpanan data (`INSERT`) ke database PostgreSQL menggunakan teknik **Prepared Statements**.

2.3.1. 3.1 Eliminasi Kerentanan SQL Injection dengan Prepared Statements

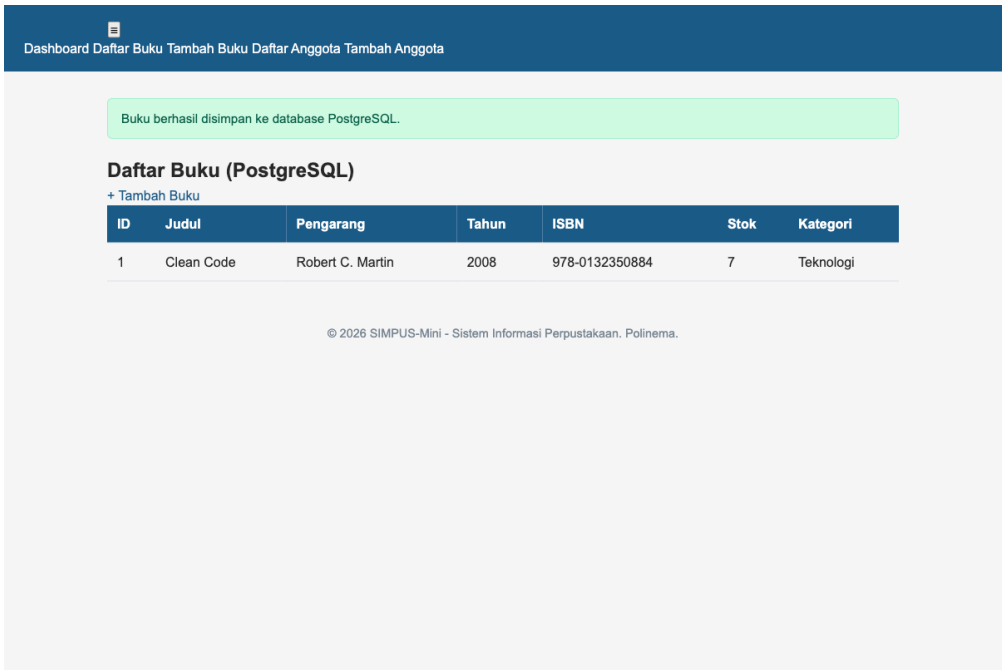
Penyimpanan data pada `books/proses_tambah.php` menggunakan query berparameter `:placeholder`:

```
$stmt = $pdo->prepare("INSERT INTO buku (judul, pengarang, tahun, isbn, stok,
kategori)
                       VALUES (:judul, :pengarang, :tahun, :isbn, :stok, :kategori)");
$stmt->execute([
    ':judul' => $judul,
    ':pengarang' => $pengarang,
    ':tahun' => (int) $tahun,
    ':isbn' => $isbn,
    ':stok' => (int) $stok,
    ':kategori' => $kategori,
]);
```

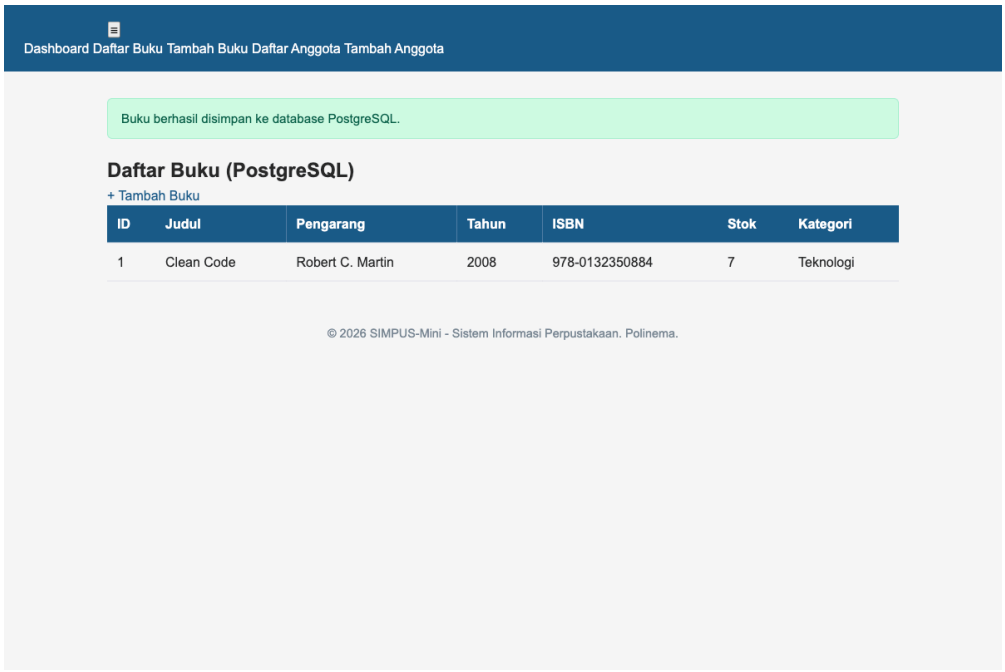
Dengan metode ini, struktur SQL dikompilasi terlebih dahulu di engine PostgreSQL, sedangkan nilai masukan dikirim secara terpisah sebagai data literal mentah. Hal ini mengeliminasi risiko serangan **SQL Injection** secara mutlak.

2.3.2. 3.2 Pembacaan Data dan Perenderan Tabel

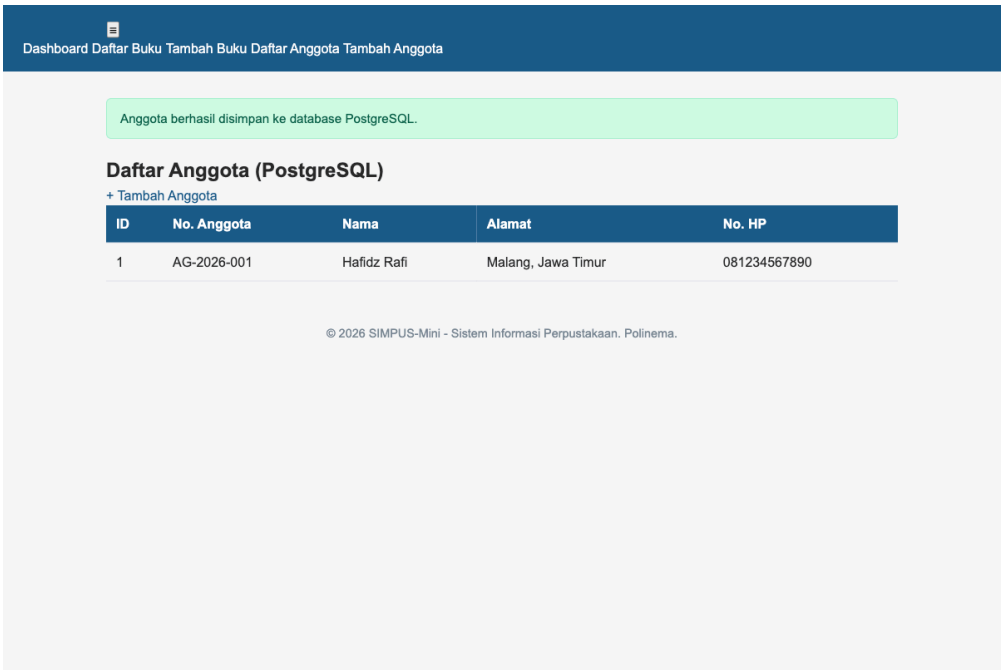
Pada `books/list.php`, data dibaca menggunakan `$stmt = $pdo->query("SELECT * FROM buku ORDER BY id DESC")` dan dirender secara dinamis:



Gambar 1.3: Notifikasi Berhasil Menyimpan Data Buku ke Database PostgreSQL Pasca-Redirect HTTP.



Gambar 1.4: Tabel Daftar Buku Berhasil Menampilkan Rekaman Persisten dari Tabel PostgreSQL.



Gambar 1.5: Tabel Daftar Anggota Berhasil Memuat Data Anggota Persisten dari Database.

2.3.3. Langkah-langkah

1. Memodifikasi `proses_tambah.php` untuk mengeksekusi prepared statements INSERT dengan parameter binding.
2. Memodifikasi `list.php` untuk membaca data dari tabel database via `query()->fetchAll()`.
3. Menguji penambahan buku baru dan memeriksa keberadaan data di database PostgreSQL.
4. Menguji penambahan anggota baru dan memverifikasi perenderan data pada modul anggota.

2.3.4. Pertanyaan

1. Bagaimana mekanisme Prepared Statements menggagalkan manipulasi SQL berbahaya?
    - Prepared statements memisahkan sintaks instruksi SQL dari nilai data masukan. Kompilasi kerangka query telah selesai sebelum data diinjeksikan. Nilai masukan hanya diperlakukan murni sebagai string literal teks, sehingga karakter kutip atau operator SQL berbahaya (seperti `' OR 1=1 --`) tidak akan pernah ditafsirkan sebagai perintah oleh database parser.

3. Tugas

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 8

Berikut adalah evaluasi kritis dan analisis mendalam terhadap lima pertanyaan evaluasi konsep pada Jobsheet 8:

No	Pertanyaan dan Jawaban Konseptual
1	**Soal:** Mengapa arsitektur penyimpanan data aplikasi SIMPUS-Mini harus dimigrasikan dari `$_SESSION` (pada Jobsheet 7) ke sistem manajemen basis data relasional seperti PostgreSQL (pada Jobsheet 8)? **Jawaban:** Variabel superglobal `$_SESSION` beroperasi di dalam memori sementara server yang terisolasi per masing-masing sesi pengguna. Data sesi tersebut akan langsung musnah ketika jendela peramban ditutup atau proses server web dimatikan, serta tidak memungkinkan kolaborasi data antar-pengguna yang berbeda. Migrasi ke PostgreSQL menyediakan persistensi data permanen di media penyimpanan sekunder, mendukung konkurensi transaksi multi-pengguna yang konsisten, memungkinkan pembentukan relasi data terstruktur, serta menjamin keandalan data transaksi sesuai kaidah ACID (**Atomicity, Consistency, Isolation, Durability**).
2	**Soal:** Apa fungsi dan keunggulan menggunakan PHP Data Objects (PDO) sebagai pustaka koneksi basis data dibandingkan menggunakan fungsi pustaka spesifik seperti `pg_connect()` atau `mysqli_connect()`? **Jawaban:** PDO bertindak sebagai lapisan abstraksi akses basis data (**Database Abstraction Layer**) terpadu di lingkungan PHP. Keunggulan utamanya mencakup: 1. **Portabilitas Kode**: Pengembang menggunakan antarmuka pemanggilan metode yang konsisten (`prepare()`, `execute()`, `fetch()`) untuk berkomunikasi dengan belasan jenis RDBMS

No	Pertanyaan dan Jawaban Konseptual
	berbeda. Jika di masa depan sistem bermigrasi ke DBMS lain, kode query aplikasi tidak perlu dirombak total. 2. **Keamanan Bawaan**: PDO mendukung **Prepared Statements** berparameter secara native untuk mitigasi SQL Injection. 3. **Manajemen Galat Modern**: Mendukung penanganan galat terstruktur berbasis pengecualian objek `PDOException`.
3	**Soal:** Jelaskan fungsi dari konfigurasi atribut `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION` dan `PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC` pada file `koneksi.php`! **Jawaban:** Konfigurasi `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION` menginstruksikan PDO untuk melempar objek pengecualian `PDOException` secara otomatis setiap kali query mengalami kegagalan sintaksis atau koneksi terputus. Hal ini memungkinkan isolasi error yang rapi menggunakan blok `try...catch`. Sementara itu, `PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC` mengatur agar hasil pembacaan baris rekaman data secara baku dikembalikan sebagai array asosiatif (nama kolom sebagai kunci array, misal `$buku['judul']`), mengeliminasi duplikasi data array numerik serta menghemat alokasi memori runtime.
4	**Soal:** Bagaimana teknik Prepared Statements (`$stmt->prepare()` dan `$stmt->execute()`) mengeliminasi risiko serangan SQL Injection secara mutlak? **Jawaban:** Pada penggabungan string query biasa, input pengguna disatukan langsung ke dalam teks perintah SQL sehingga karakter jahat dapat membelokkan logika perintah. Dengan Prepared Statements, siklus eksekusi dipisahkan menjadi dua tahapan ketat:

No	Pertanyaan dan Jawaban Konseptual
	1. **Fase Kompilasi (Prepare)**: Struktur perintah SQL dikirim dan dikompilasi terlebih dahulu oleh parser database dengan placeholder parameter (`:parameter`). 2. **Fase Pengikatan Data (Execute)**: Nilai masukan pengguna dikirim secara terpisah murni sebagai nilai data literal mentah. Database engine tidak akan pernah menafsirkan karakter kutip atau ekspresi logika tambahan sebagai perintah SQL, menggagalkan segala bentuk eksploitasi injeksi kode.
5	**Soal:** Jelaskan peran integritas data dari klausa `SERIAL PRIMARY KEY` pada tabel buku dan klausa `NOT NULL UNIQUE` pada kolom `no_anggota` di skema `01_buku_anggota.sql`! **Jawaban:** Klausa `SERIAL PRIMARY KEY` pada PostgreSQL secara otomatis mendefinisikan generator sequence angka urut (**auto-increment**) yang menghasilkan bilangan bulat unik bagi setiap baris rekaman baru, sekaligus menetapkannya sebagai kunci utama tak-boleh-kosong (**Primary Key**) yang menjadi pengenal unik entitas. Klausa `NOT NULL UNIQUE` pada `no_anggota` menegakkan integritas domain: `NOT NULL` mewajibkan setiap anggota memiliki nomor identitas keanggotaan terdaftar, sedangkan `UNIQUE` menginstruksikan database engine untuk menolak pencatatan nomor anggota duplikat, menjamin keunikan identitas anggota di seluruh sistem.